

Atualizacao de progresso BullMQ em fluxos reais

Contexto

Este documento complementa dois materiais anteriores:

- docs/Redis_BullMQ_Queue/explicacao_fundacao_bullmq.md
- docs/Redis_BullMQ_Queue/plano_escalabilidade_redis_bullmq.md

Na documentacao anterior, a conclusao era: a fundacao da fila existia, mas os fluxos reais ainda nao tinham sido migrados.

O avanço de hoje foi justamente sair dessa etapa.

Agora o sistema nao esta mais apenas com a fundacao pronta. Ele ja possui fluxos reais passando pela fila em producao, com API web atuando como producer e worker separado atuando como executor.

Resumo executivo

O que ficou validado hoje:

- o worker separado esta de pe no Render e consumindo jobs reais
- a API web esta de pe no Render e respondendo corretamente como producer
- o fluxo otp-cleanup foi migrado e validado ponta a ponta
- o fluxo subscription-generation foi migrado e validado ponta a ponta
- o cron e a URL dos endpoints internos nao precisaram mudar para esse primeiro corte
- a regra de negocio do subscription continuou exatamente a mesma, inclusive seus erros esperados de dominio

Em uma frase:

Antes, a fila existia, mas ainda nao era usada por fluxos reais.

Agora, a fila ja esta sendo usada por dois fluxos internos reais.

O que avancou em relacao aos documentos anteriores

Estado descrito em explicacao_fundacao_bullmq.md

O documento da fundacao dizia, corretamente, que:

- a fila estava criada
- o worker estava criado
- os handlers antigos ja podiam ser reaproveitados
- os fluxos reais ainda nao tinham sido conectados

Hoje esse ponto mudou.

Os fluxos abaixo ja foram conectados a BullMQ:

- otp-cleanup
- subscription-generation

Estado descrito em plano_escalabilidade_redis_bullmq.md

Pelo plano logico, o sistema estava pronto para sair da fase de fundacao e entrar na migracao progressiva de jobs internos.

Com o que foi feito hoje, o status passa a ser este:

- Fase 2: fundacao BullMQ e worker dedicado validada
- Fase 3: iniciada e com dois fluxos reais internos tratados

Leitura mais precisa da fase atual:

- otp-cleanup: migrado e validado ponta a ponta
- subscription-generation: migrado e validado ponta a ponta no aspecto de infraestrutura e execucao
- subscription-expiry: ainda nao migrado neste ciclo
- pagamentos e webhooks: ainda fora do escopo

O que foi feito hoje

1. Ajuste operacional da API web no Render

Foi corrigido o bind do servidor HTTP para usar host 0.0.0.0 no Render.

Isso resolveu o problema anterior em que a API chegava a subir o processo, mas o Render nao conseguia detectar a porta corretamente.

Resultado:

- health e readiness passaram a responder corretamente
- a API ficou estavel para os testes remotos

2. Validacao real do fluxo otp-cleanup

O fluxo otp-cleanup foi validado em dois modos.

Com a flag desligada

A API executou o job de forma sincrona dentro da propria request.

Exemplo de resposta:

```
{"ok":true,"expired":1,"durationMs":409}
```

Com a flag ligada

A API deixou de executar o job diretamente e passou a enfileira-lo.

Exemplo de resposta:

```
{ "ok": true, "enqueued": true, "jobId": "1", "correlationId": "016ab921-4184-4742-a770-8b2de7e27e7a", "durationMs": 102 }
```

No worker, os logs confirmaram:

- Internal job started
- Internal job completed

com o mesmo jobId e o mesmo correlationId.

Conclusao do otp-cleanup:

- producer validado
- worker validado
- regra de negocio validada
- fallback validado anteriormente

3. Migracao da rota de subscription-generation

Foi aplicado o mesmo padrao do otp-cleanup ao endpoint interno de subscription.

O desenho ficou assim:

- com USE_BULLMQ_SUBSCRIPTION=false, a API executa runSubscriptionJob de forma sincrona
- com USE_BULLMQ_SUBSCRIPTION=true, a API apenas enfileira o job
- se o enqueue falhar, a API cai no fallback sincrono

Importante:

- o endpoint permaneceu o mesmo
- a autenticacao interna permaneceu a mesma
- o worker ja estava preparado para consumir subscription-generation
- o cron nao precisou mudar

4. Validacao real do fluxo subscription-generation

Assim como no otp-cleanup, o fluxo de subscription foi validado em dois modos.

Com a flag desligada

A resposta voltou no formato sincrono esperado:

```
{ "ok": true, "processed": 1, "created": 0, "failed": 1, "durationMs": 700 }
```

O worker nao processou esse job nessa etapa, como esperado.

Com a flag ligada

A resposta passou a vir como producer:

```
{ "ok": true, "enqueued": true, "jobId": "2", "correlationId": "132faa15-4956-4bf5-a9f3-ef9957a8640d", "durationMs": 103 }
```

No worker, os logs confirmaram:

- Internal job started
- jobName: subscription-generation
- jobId: 2
- correlationId: 132faa15-4956-4bf5-a9f3-ef9957a8640d
- Internal job completed

O que isso prova tecnicamente

O que ficou provado hoje nao e apenas que a fila existe.

Ficou provado que:

1. a API web consegue publicar jobs reais no Redis via BullMQ
2. o worker separado consegue consumir esses jobs reais
3. a correlacao entre request e job esta funcionando
4. o rollout por flag esta funcionando
5. o rollback continua simples, baseado apenas em variavel de ambiente
6. o cron atual consegue continuar usando a mesma URL, sem mudanca de topologia

Em termos de arquitetura, o sistema deixou de estar apenas na fase "fundacao pronta" e passou para "primeiros fluxos reais usando a fila".

Ponto importante sobre subscription

O fluxo de subscription gerou erro de negocio tanto no modo sincrono quanto no modo assincrono.

O erro observado foi:

- ScheduleServiceError
- code: LEAD_TIME_VIOLATION
- message: Janela de entrega requer antecedencia minima de 2h

Esse ponto e importante porque ele mostra que:

- a migracao para fila nao alterou a regra de negocio
- o worker esta executando a mesma logica que antes rodava dentro da API
- o erro nao e da infraestrutura BullMQ
- o erro nao e de Redis
- o erro nao e do worker

Ou seja:

o comportamento de negocio foi preservado.

No caso do teste manual feito fora da janela operacional ideal, o job falhou exatamente como falharia antes.

Isso nao invalida a migracao tecnica. Pelo contrario, prova que a fila apenas mudou o modo de execucao, e nao a regra do dominio.

O que nao mudou ainda

Mesmo com o avanço de hoje, estas partes ainda nao foram migradas:

- subscription-expiry
- pagamentos
- webhooks
- conciliacao financeira
- Socket.IO com adapter Redis
- dashboard operacional de fila
- DLQ dedicada e estrategia operacional de replay mais avancada

Tambem nao houve mudanca, ate aqui, em:

- URL dos crons
- INTERNAL_JOB_SECRET
- topologia manual dos servicos no Render
- separacao entre servico web e servico worker

Arquivos e pontos principais envolvidos no avanço de hoje

Arquivos centrais da infraestrutura e dos fluxos migrados:

- apps/api/src/server/index.ts
- apps/api/src/infra/queue/contracts.ts
- apps/api/src/infra/queue/internal-jobs.producer.ts
- apps/api/src/worker/index.ts
- apps/api/src/worker/processors/internal-jobs.processor.ts
- apps/api/src/jobs/otp-cleanup-dispatch.ts
- apps/api/src/jobs/subscription-dispatch.ts
- apps/api/src/jobs/jobs.routes.ts
- apps/api/src/jobs/otp-cleanup-job.ts
- apps/api/src/jobs/subscription-job.ts

Flags introduzidas para rollout:

- USE_BULLMQ_OTP_CLEANUP
- USE_BULLMQ_SUBSCRIPTION

Status atual recomendado para o time

Leitura objetiva do estado atual:

- BullMQ ja nao e mais apenas fundacao
- o worker ja participa do sistema real
- otp-cleanup pode ser considerado migrado com sucesso
- subscription-generation pode ser considerado migrado com sucesso no aspecto tecnico
- subscription-generation ainda precisa ser acompanhado do ponto de vista de regra de agenda e horario de execucao

Proximos passos recomendados

1. Observar o proximo disparo real do cron de subscription

Como o teste manual foi feito fora do horario ideal, o proximo passo mais util e observar o comportamento do cron real no horario operacional esperado.

O objetivo aqui nao e validar a fila, porque isso ja foi validado.

O objetivo agora e validar o comportamento de negocio no horario correto.

2. Migrar subscription-expiry com o mesmo padrao

O proximo candidato natural e subscription-expiry.

Motivos:

- ainda e job interno
- segue o mesmo desenho de endpoint protegido
- aproveita a mesma infraestrutura ja validada
- mantem o plano de migracao progressiva antes de tocar em pagamentos

3. Nao tocar em pagamentos e webhooks ainda

O plano original continua correto nesse ponto.

Agora que otp-cleanup e subscription-generation ja provaram a infraestrutura, o proximo degrau ainda deve ficar no campo de jobs internos e nao financeiros.

Pagamentos e webhooks continuam devendo entrar depois.

4. Tratar a questao de lead time como tema de negocio, nao de fila

O erro de subscription observado hoje nao pede correcoes em BullMQ.

Ele pede uma decisao funcional sobre uma destas possibilidades:

- manter a regra como esta e aceitar que testes manuais fora da janela falhem
- criar uma estrategia de dry-run ou staging para esse tipo de validacao
- revisar se o cron real esta rodando no horario correto para o modelo de lead time esperado

Conclusao

O avanço de hoje foi relevante.

O projeto saiu do estado "a fila existe, mas ainda nao esta sendo usada por fluxos reais" e passou para um estado novo:

- a fila esta em uso real
- a API publica jobs reais
- o worker consome jobs reais
- o rollout por flag foi testado em producao
- o comportamento do dominio foi preservado

Em resumo:

na documentacao anterior, a frase correta era "Criamos a fila. Agora falta usar a fila."

Depois do que foi validado hoje, a frase passa a ser:

"Ja estamos usando a fila em fluxos internos reais. O proximo passo e ampliar isso com controle, sem pular direto para pagamentos."